디지털 서명 시스템

개요

CometBFT의 디지털 서명 시스템은 블록체인의 보안과 무결성을 보장하는 핵심 구성 요소입니다. 이 시스템은 밸리데이터들이 블록과 투표에 서명하고, 네트워크가 이러한 서명들을 검증하여 합의에 도달하는 과정을 관리합니다.

CometBFT의 서명 시스템은 다음과 같은 주요 기능을 제공합니다:

- **밸리데이터 서명**: 블록 제안과 투표에 대한 디지털 서명
- **서명 검증**: 개별 서명과 배치 서명의 효율적인 검증
- **서명 집계**: 다중 서명의 효율적인 처리
- **이중 서명 방지**: 악의적인 밸리데이터의 이중 서명 탐지

핵심 개념

서명 유형

CometBFT에서 사용되는 주요 서명 유형들:

1. **투표 서명 (Vote Signature)**

- Prevote와 Precommit 투표에 대한 서명
- 합의 과정에서 밸리데이터의 의견 표명

2. **블록 제안 서명 (Proposal Signature)**

- 블록 제안자가 생성하는 서명
- 제안된 블록의 유효성 보장

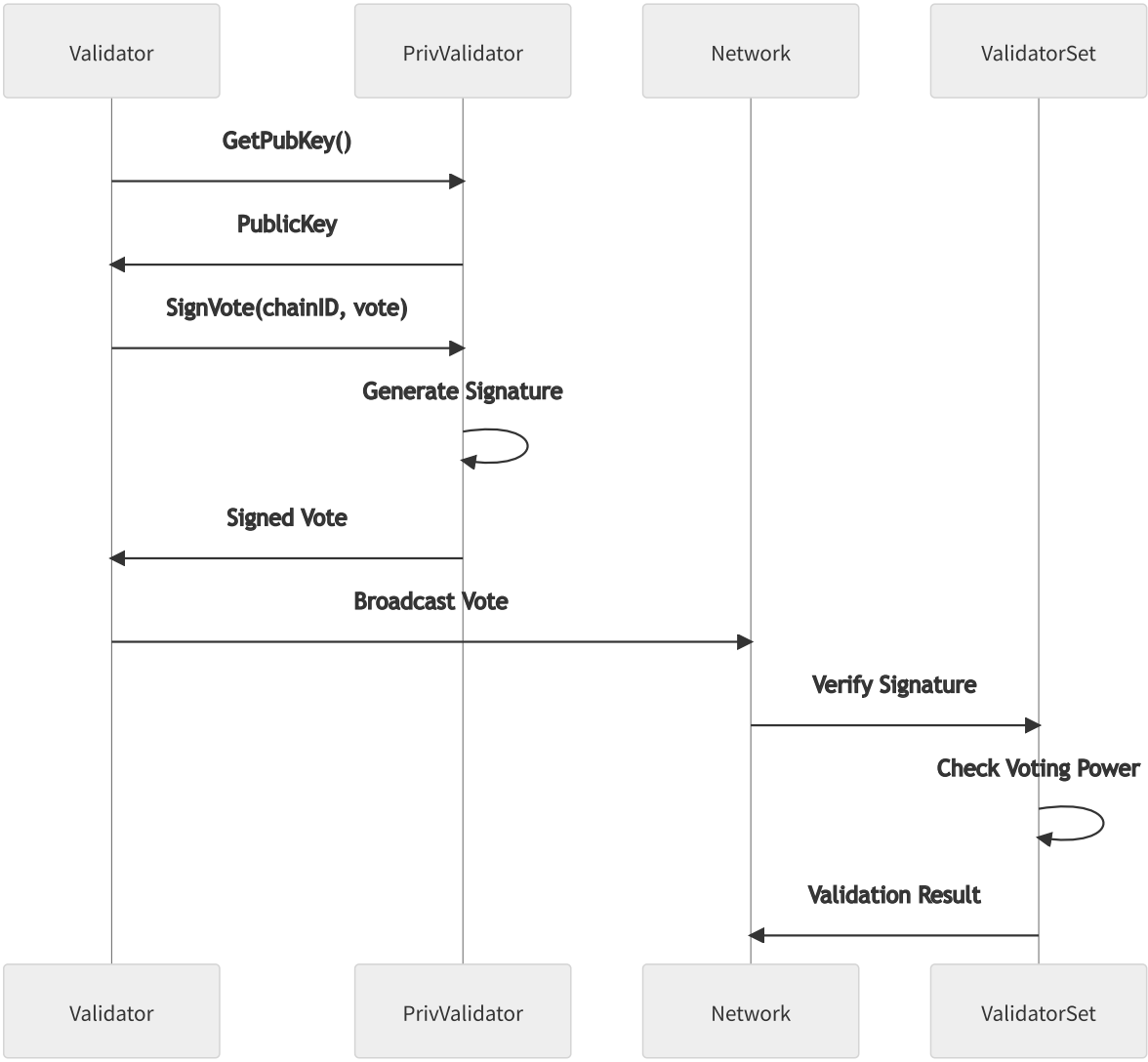
3. **확장 서명 (Extension Signature)**

- Vote Extension에 대한 추가 서명
- 애플리케이션별 확장 데이터 보호

4. **비재생 방지 확장 서명 (Non-Replay Extension Signature)**

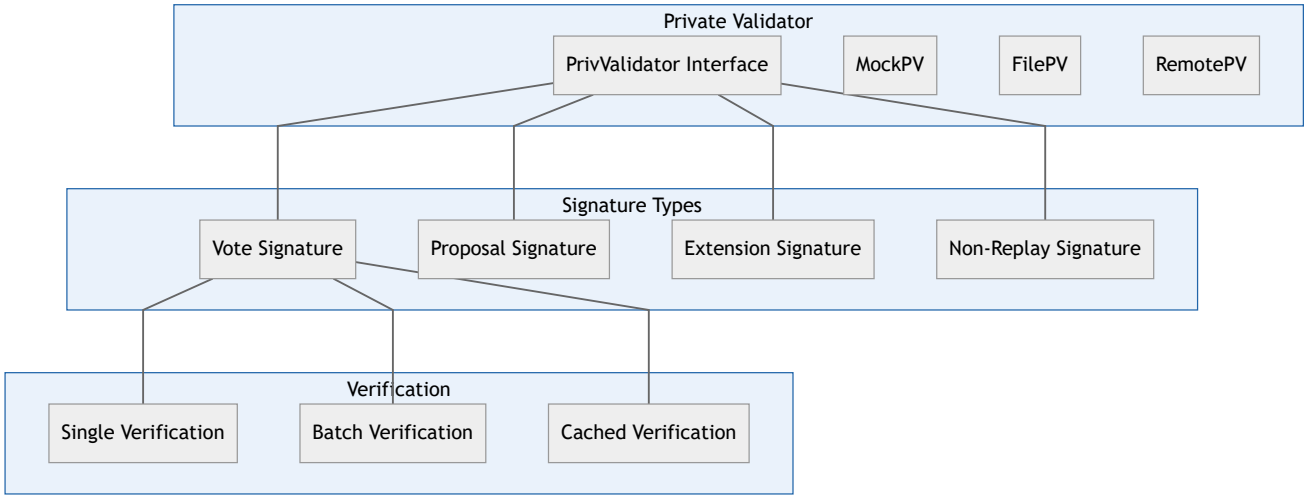
- 재생 공격 방지를 위한 추가 서명
- 네트워크 간 서명 재사용 방지

서명 생명주기



아키텍처

서명 시스템 구조



PrivValidator 인터페이스

```
type PrivValidator interface {
// 공개키 반환
GetPubKey() (crypto.PubKey, error)
// 투표 서명 (확장 서명 포함)
SignVote(chainID  string, vote *cmtproto.Vote, signExtension  bool) error
// 블록 제안 서명
SignProposal(chainID  string, proposal *cmtproto.Proposal) error
// 임의 바이트 서명
SignBytes(bytes []byte) ([]byte, error)
}
```

밸리데이터 서명 생성

투표 서명 과정

투표 서명은 CometBFT 합의 과정의 핵심입니다:

```
// 투표 서명 생성
func (pv MockPV) SignVote(chainID  string, vote *cmtproto.Vote, signExtension  b
// 1. 투표 서명 바이트 생성
signBytes := VoteSignBytes(chainID, vote)
// 2. 기본 투표 서명
sig, err := pv.PrivKey.Sign(signBytes)
if  err != nil {
return  err
}
vote.Signature = sig
// 3. 확장 서명 처리 (Precommit이고 non-nil인 경우)
if  signExtension {
if  vote.Type == PrecommitType && !ProtoBlockIDIsNil(&vote.BlockID) {
// 확장 서명 바이트 생성
extSignBytes, nonRpExtSignBytes := VoteExtensionSignBytes(chainID, vote)
// 재생 방지 확장 서명
extSig, err := pv.PrivKey.Sign(extSignBytes)
if  err != nil {
```

```go
return err
}
// 비재생 방지 확장 서명
nonRpExtSig, err := pv.PrivKey.Sign(nonRpExtSignBytes)
if err != nil {
return err
}
vote.ExtensionSignature = extSig
vote.NonRpExtensionSignature = nonRpExtSig
}
}
return nil
}
```

서명 바이트 생성

서명을 위한 정규화된 바이트 생성:

```go
// 투표 서명 바이트 생성
func VoteSignBytes(chainID string, vote *cmtproto.Vote) []byte {
// 정규화된 투표 생성
pb := CanonicalizeVote(chainID, vote)
// Protobuf 직렬화 (varint 길이 접두사 포함)
bz, err := protoio.MarshalDelimited(&pb)
```

```go
if err != nil {
panic(err)
}
return bz
}

// 확장 서명 바이트 생성
func VoteExtensionSignBytes(chainID string, vote *cmtproto.Vote) ([]byte, []by
// 재생 방지 확장 서명 바이트
pb := CanonicalizeVoteExtension(chainID, vote)
bz, err := protoio.MarshalDelimited(&pb)
if err != nil {
panic(err)
}
// 비재생 방지 확장 서명 바이트
bz2 := vote.NonRpExtension
return bz, bz2
}
```

블록 제안 서명

블록 제안자의 서명 생성 과정:

```go
func (pv MockPV) SignProposal(chainID string, proposal *cmtproto.Proposal) erro
```

```go
// 제안 서명 바이트 생성
signBytes := ProposalSignBytes(chainID, proposal)
// 서명 생성
sig, err := pv.PrivKey.Sign(signBytes)
if err != nil {
return err
}
proposal.Signature = sig
return nil
}
```

서명 검증 과정

개별 서명 검증

단일 투표의 서명 검증:

```go
// 기본 투표 검증
func (vote *Vote) Verify(chainID string, pubKey crypto.PubKey) error {
// 1. 주소 검증
if !bytes.Equal(pubKey.Address(), vote.ValidatorAddress) {
return ErrVoteInvalidValidatorAddress
}
// 2. 서명 바이트 생성
```

```go
v := vote.ToProto()
signBytes := VoteSignBytes(chainID, v)
// 3. 서명 검증
if !pubKey.VerifySignature(signBytes, vote.Signature) {
return ErrVoteInvalidSignature
}
return nil
}


// 확장 서명 포함 검증
func (vote *Vote) VerifyVoteAndExtension(chainID string, pubKey crypto.PubKey)
// 기본 투표 검증
v, err := vote.verifyAndReturnProto(chainID, pubKey)
if err != nil {
return err
}
// 확장 서명 검증 (non-nil precommit인 경우)
if vote.Type == PrecommitType && !ProtoBlockIDIsNil(&v.BlockID) {
if len(vote.ExtensionSignature) == 0 || len(vote.NonRpExtensionSignature) == 0
return ErrVoteNoSignature
}
// 확장 서명 바이트 생성 및 검증
extSignBytes, nonRpExtSignBytes := VoteExtensionSignBytes(chainID, v)
```

```
if !pubKey.VerifySignature(extSignBytes, vote.ExtensionSignature) {
return ErrVoteInvalidSignature
}
if !pubKey.VerifySignature(nonRpExtSignBytes, vote.NonRpExtensionSignature) {
return ErrVoteInvalidSignature
}
}
return nil
}
```

배치 서명 검증

효율적인 다중 서명 검증:

```
// 배치 검증기 생성
func CreateBatchVerifier(pk crypto.PubKey) (crypto.BatchVerifier, bool) {
switch pk.Type() {
case ed25519.KeyType:
return ed25519.NewBatchVerifier(), true
default:
return nil, false
}
}
```

```go
// 배치 검증 사용 예제
func verifyCommitBatch(validators []*Validator, votes []*Vote, chainID string)
// Ed25519 배치 검증기 생성
batchVerifier := ed25519.NewBatchVerifier()
for i, vote := range votes {
if vote == nil {
continue
}
validator := validators[i]
signBytes := VoteSignBytes(chainID, vote.ToProto())
// 배치에 추가
err := batchVerifier.Add(validator.PubKey, signBytes, vote.Signature)
if err != nil {
return err
}
}
// 배치 검증 실행
ok, results := batchVerifier.Verify()
if !ok {
return errors.New("batch verification failed")
}
return nil
```

```
}
```

서명 집계 및 임계값 서명

투표 집계

밸리데이터 세트에서의 투표 집계:

```
// 커밋 검증 (2/3+ 투표 파워 확인)
func VerifyCommit(chainID string, vals *ValidatorSet, blockID BlockID,
height int64, commit *Commit) error {
if vals.Size() != len(commit.Signatures) {
return fmt.Errorf("invalid commit signatures count")
}
talliedVotingPower := int64(0)
votingPowerNeeded := vals.TotalVotingPower() * 2 / 3
for idx, commitSig := range commit.Signatures {
if commitSig.Absent() {
continue
}
// 밸리데이터 검증
val := vals.Validators[idx]
// 투표 재구성
vote := &Vote{
```

```
Type: PrecommitType,
Height: height,
Round: commit.Round,
BlockID: blockID,
Timestamp: commitSig.Timestamp,
ValidatorAddress: commitSig.ValidatorAddress,
ValidatorIndex: int32(idx),
Signature: commitSig.Signature,
}
// 서명 검증
if err := vote.Verify(chainID, val.PubKey); err != nil {
return err
}
talliedVotingPower += val.VotingPower
}
// 2/3+ 투표 파워 확인
if talliedVotingPower <= votingPowerNeeded {
return ErrNotEnoughVotingPowerSigned{
Got: talliedVotingPower,
Needed: votingPowerNeeded + 1,
}
}
return nil
```

```
}
```

경량 클라이언트 검증

효율적인 부분 검증:

```
// 경량 검증 (모든 서명을 검증하지 않음)
func VerifyCommitLight(chainID string, vals *ValidatorSet, blockID BlockID,
height int64, commit *Commit) error {
talliedVotingPower := int64(0)
votingPowerNeeded := vals.TotalVotingPower() * 2 / 3
for idx, commitSig := range commit.Signatures {
if commitSig.Absent() {
continue
}
val := vals.Validators[idx]
// 투표 파워 누적
talliedVotingPower += val.VotingPower
// 필요한 투표 파워에 도달하면 조기 종료
if talliedVotingPower > votingPowerNeeded {
// 마지막 서명만 검증
vote := reconstructVote(height, commit.Round, blockID, commitSig, idx)
return vote.Verify(chainID, val.PubKey)
}
```

```
}

return ErrNotEnoughVotingPowerSigned{

Got: talliedVotingPower,

Needed: votingPowerNeeded + 1,

}

}
```

이중 서명 방지

이중 서명 탐지

악의적인 밸리데이터의 이중 서명 탐지:

```
// 충돌하는 투표 오류

type ErrVoteConflictingVotes struct {

VoteA *Vote

VoteB *Vote

}


func (err *ErrVoteConflictingVotes) Error() string {

return  fmt.Sprintf("conflicting votes from validator %X", err.VoteA.ValidatorAd

}


// 이중 서명 검사
```

```go
func detectDoubleVoting(votes []*Vote) error {
votesByValidator := make(map[string]*Vote)
for _, vote := range votes {
key := fmt.Sprintf("%X-%d-%d", vote.ValidatorAddress, vote.Height, vote.Round)
if existingVote, exists := votesByValidator[key]; exists {
// 동일한 밸리데이터의 다른 투표 발견
if !vote.BlockID.Equals(existingVote.BlockID) {
return NewConflictingVoteError(existingVote, vote)
}
} else {
votesByValidator[key] = vote
}
}
return nil
}
```

안전한 서명 생성

이중 서명 방지를 위한 안전 장치:

```go
// 안전한 투표 서명
func SignAndCheckVote(vote *Vote, privVal PrivValidator, chainID string,
extensionsEnabled bool) (bool, error) {
// 1. 서명 생성
```

```go
v := vote.ToProto()
if err := privVal.SignVote(chainID, v, extensionsEnabled); err != nil {
return true, err // 복구 가능한 오류
}
vote.Signature = v.Signature
// 2. 서명 유효성 검사
isPrecommit := vote.Type == PrecommitType
isNil := vote.BlockID.IsNil()
extSignature := (len(v.ExtensionSignature) > 0)
nonRpExtSignature := (len(v.NonRpExtensionSignature) > 0)
// 3. 확장 서명 검증
if (extSignature || nonRpExtSignature) && (!isPrecommit || isNil) {
return false, &ErrVoteExtensionInvalid{
Reason: "vote extension signature must not be present in prevotes or nil-precomm
ExtSignature: v.ExtensionSignature,
}
}
// 4. 필수 확장 서명 검사
if extensionsEnabled && isPrecommit && !isNil {
if !extSignature || !nonRpExtSignature {
return false, &ErrVoteExtensionInvalid{
Reason: "vote extension signature must be present if extensions are enabled",
ExtSignature: v.ExtensionSignature,
```

```
}
}
}
// 5. 서명 할당
vote.ExtensionSignature = v.ExtensionSignature
vote.NonRpExtensionSignature = v.NonRpExtensionSignature
vote.Timestamp = v.Timestamp
return true, nil
}
```

성능 최적화

서명 캐싱

반복적인 서명 검증 최적화:

```
// 서명 캐시 인터페이스
type SignatureCache interface {
Get(signature, pubkey, signBytes []byte) (bool, bool)
Set(signature, pubkey, signBytes []byte, valid  bool)
}

// 캐시를 활용한 검증
func  VerifyCommitLightWithCache(chainID  string, vals *ValidatorSet,
```

```
blockID BlockID, height  int64, commit *Commit,
cache SignatureCache) error {
for  idx, commitSig := range  commit.Signatures {
if  commitSig.Absent() {
continue
}
val := vals.Validators[idx]
vote := reconstructVote(height, commit.Round, blockID, commitSig, idx)
signBytes := VoteSignBytes(chainID, vote.ToProto())
// 캐시 확인
if  cached, valid := cache.Get(vote.Signature, val.PubKey.Bytes(), signBytes);
if !valid {
return  ErrVoteInvalidSignature
}
continue
}
// 서명 검증
valid := val.PubKey.VerifySignature(signBytes, vote.Signature)
// 캐시에 저장
cache.Set(vote.Signature, val.PubKey.Bytes(), signBytes, valid)
if !valid {
return  ErrVoteInvalidSignature
}
```

```
}
return nil
}
```

배치 검증 최적화

Ed25519 배치 검증 활용:

```
// 최적화된 커밋 검증
func optimizedVerifyCommit(vals *ValidatorSet, commit *Commit, chainID string)
// 모든 키가 동일한 타입인지 확인
if !vals.AllKeysHaveSameType() {
return verifyCommitIndividually(vals, commit, chainID)
}
// Ed25519인 경우 배치 검증 사용
if vals.Validators[0].PubKey.Type() == ed25519.KeyType {
return verifyCommitBatch(vals, commit, chainID)
}
// 다른 키 타입은 개별 검증
return verifyCommitIndividually(vals, commit, chainID)
}
```

보안 고려사항

타이밍 공격 방지

서명 검증에서의 타이밍 공격 방지:

```
// 상수 시간 비교
func constantTimeCompare(a, b []byte) bool {
if len(a) != len(b) {
return false
}
var result byte
for i := 0; i < len(a); i++ {
result |= a[i] ^ b[i]
}
return result == 0
}
```

서명 가단성 방지

Secp256k1에서의 서명 가단성 방지:

```
func (pubKey PubKey) VerifySignature(msg []byte, sigStr []byte) bool {
// ... 기본 검증 ...
// 가단성 방지: 낮은 S 값 강제
modifiedSignature, parseErr := ecdsa.ParseDERSignature(signature.Serialize())
if parseErr != nil {
```

```
return false
}
// 서명이 정규화 후 변경되면 거부
if !signature.IsEqual(modifiedSignature) {
return false
}
return signature.Verify(sum[:], pub)
}
```

재생 공격 방지

체인 ID를 통한 재생 공격 방지:

```
// 체인별 서명 바이트 생성
func VoteSignBytes(chainID string, vote *cmtproto.Vote) []byte {
// 체인 ID가 서명에 포함되어 다른 체인에서 재사용 불가
pb := CanonicalizeVote(chainID, vote)
return protoio.MarshalDelimited(&pb)
}
```

CometBFT의 디지털 서명 시스템은 이러한 다층적인 보안 메커니즘을 통해 블록체인의 무결성과 합의의 신뢰성을 보장하며, 동시에 높은 성능과 확장성을 제공합니다.